Lecture 15 - Approximate Query Processing

Setup

- Another AQP: A this time is *approximate*
- Approximation is natural: want to look at less data, get most of the accuracy
  - AQP is had been around at least since the 1990s
- The paper's setting: just after MapReduce and Spark
  - Systems of that sort often took minutes to hours to query
- If you have a query `SELECT AVG(SessionTime) FROM Sessions WHERE City = _location_` and want to use approximation:
  - One extreme: online aggregation
    - Recall, we talked about this last week when discussing adaptivity
    - Basically keep sampling until your CIs "look nice"
    - Downside: biased towards popular cities
      - Example from paper: may be OK for NY, but not for Galena IL (a city with <4000 people)
  - Another extreme: sampling and sketching
    - Basically pre-built uniform random samples don't work - same reason as OLA
      - Another: build samples based on strata - which had been explored before but not as comprehensively as this paper
    - Let's take a detour to cover sampling and sketching
- Other use-cases of approximation: cardinality estimation
  - A histogram is itself an approximate query result
- Let's start by discussing sampling and sketching techniques ...

Sketching

- Sketching - focuses on building small (typically much much smaller than O(n)) structures targeted towards answering specific queries
  - also low time
- Q: Who here remembers reservoir sampling?
  - Sample of size k from a stream of unknown size
  - Flip a coin per new element, keep with probability k/n
- Main game is:
  - get an unbiased estimate
  - ensure that the variance of the estimate is low
- Lots of theory papers in this space!
- Q: Who here remembers Bloom filters? We covered this earlier.
- Count-min sketch (Cormode, Muthukrishnan, 2003)
  - Goal: *COUNT($\star$) per key
  - A Count-Min Sketch is a 2D array of counters with dimensions d × w (d hash functions, w buckets each).
  - To insert an item x:
    - For each row i, compute $h_i(x)$ and increment to the counter at position (i, $h_i(x)$).
  - To query the frequency of item x:
    - For each row i, look up the counter at (i, $h_i(x)$).
    - Return the **minimum** across all d rows.
  - Each count will be an overestimate (why?), but by taking the minimum, we end up with a provably good upper bound on true frequency
    - Overestimate because of hash collisions

    - Usually used for:
        - Finding heavy hitters
        - Join size estimation: hash both sides into sketches and take an inner product
    - Can also cleanly handle deletions
- HyperLogLog - based on Flajolet Martin 1985, with refinements since then
    - Goal: *COUNT DISTINCT*
        - But do so in much less than O(n), because that is the space you need to keep track of all items
    - Key idea: Hash each element to a binary string. If your hash function is uniform, then:
        - Half of elements will have a hash starting with `1`
        - A quarter will start with `01`
        - An eighth will start with `001`
        - etc.
    - Seeing a hash with k leading zeroes is rare: if we see something like this, we've likely seen 2^k values
    - Algorithm: keep track of maximum sequence of zeros seen so far; count is 2^m
    - Unfortunately, high variance. Improvements:
        - Multiple hashes (plus combination through harmonic mean), odds that all have this stray element is low (LogLog)
        - Treat setting of low cardinality different from high cardinality (HyperLogLog)
- Whole field of **sublinear time algorithms**
    - Don't even read through everything once!
    - But: are allowed to ask arbitrary queries

General Sampling Approaches

- Consider a single-table SQL query with an arbitrary WHERE clause.
- From a population of m items and a sample of size n < m, you can compute a scaled estimate of many aggregate functions in a natural way:
    - avg ~= sum-so-far/n
    - sum ~= m *avg* = *sum-so-far* m / n
    - count- ~= count-so-far * m / n
- How good is that estimate? Intuitively, a bigger sample should give a “tighter” estimate.
    - Need a **confidence interval** for the estimator. That is:
        - Given: a confidence parameter p between 0 and 1, we want to provide a precision parameter ϵ_n such that the estimator is $\pm\epsilon_n$ of the final answer with probability approximately equal to p. There are various statistical formulae to compute the confidence interval:
    - **Conservative confidence intervals**, e.g. based on Hoeffding’s inequality, guarantee that the precision parameter is true with probability p.
        - But the resulting intervals can be pretty wide.
        - Hoeffding requires knowing the range of the random variable (max - min), which is often unknown or very large, making bounds loose.
    - **Large-sample confidence intervals**, e.g. based on the Central Limit Theorem, guarantee the precision parameter is true with probability that is approximately p.
        - In practice these are used in statistics with very modest sample sizes (dozens of items), but it’s hard to be sure when they “kick in”.
        - CLT: when sampling *n* items with replacement, the average $\hat{\mu}$ of the items so far has approximately a normal distribution with mean being the true answer μ and variance σ^2/n.
    - Another option: **boostrapping**: resample from your sample to estimate variance empirically
- Old history for sampling in the database context: Frank Olken's thesis at UC Berkeley 1993
    - PM at NSF for many years

    - Considered sampling through index structures, with and w/o replacement
- Q: Sampling from joins is hard, why?
    - Can't simply take a sample from each, and then "hope for the best"
    - Why does this not work?
        - Renumbering "each side" based on the samples taken, this ends up being a "corner"
        - Clearly not an unbiased sample of the join, because it's concentrated at the corner
        - So overall: **The join of two samples is not a sample of the join**; need to debias
    - Olken's approach: sample from one side, do complete join, then sample from that result as appropriate
    - Another way to debias: ripple joins (remember that?)
- Reminder and Q: what is stratified sampling?
    - Going back to our query `SELECT AVG(SessionTime) FROM Sessions WHERE City = _location_`
    - Want to keep samples for each location: oversample Galena, undersample NY relative to uniform

The BlinkDB paper: Motivation

- Want *efficiency* (like sketch-based approaches) and *generality* (like OLA)
- Want to support *response time bounds* or *error bounds*
- Prior work had already suggested sampling along strata, but was specific to certain SQL query templates: **not general, but compact**
- What if you sample across all combinations of attributes across all relations
    - Exponential very quickly; will end up being O(data size)
    - So-called "curse-of-dimensionality"
    - **not compact, but general**
- BlinkDB instead makes the claim: defining strata along **query column sets** offers a **nice balance of generality and compactness**
    - Based on workload patters from Facebook and Conviva
- Two main ideas:
    - sample construction along QCSs
        - given past usage of QCSs and a sample materialization space budget, pick QCSs to materialize
    - sample selection from QCSs
        - can itself run query on subsamples of existing QCSs to choose the best sample to use
        - uses a so-called ELP (Error-Latency Profile)
- Implemented on top of Shark - precursor to Spark SQL

Background and QCS motivation

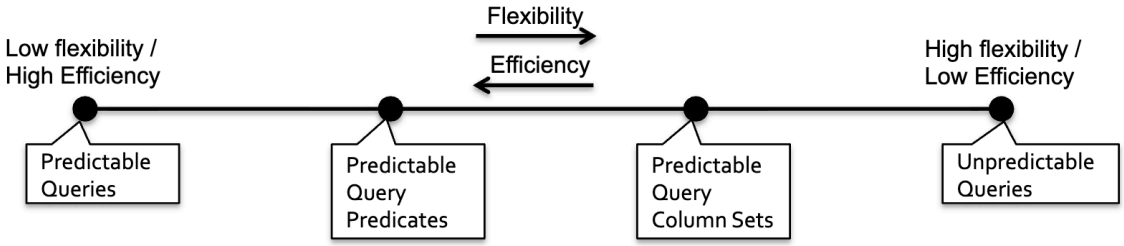
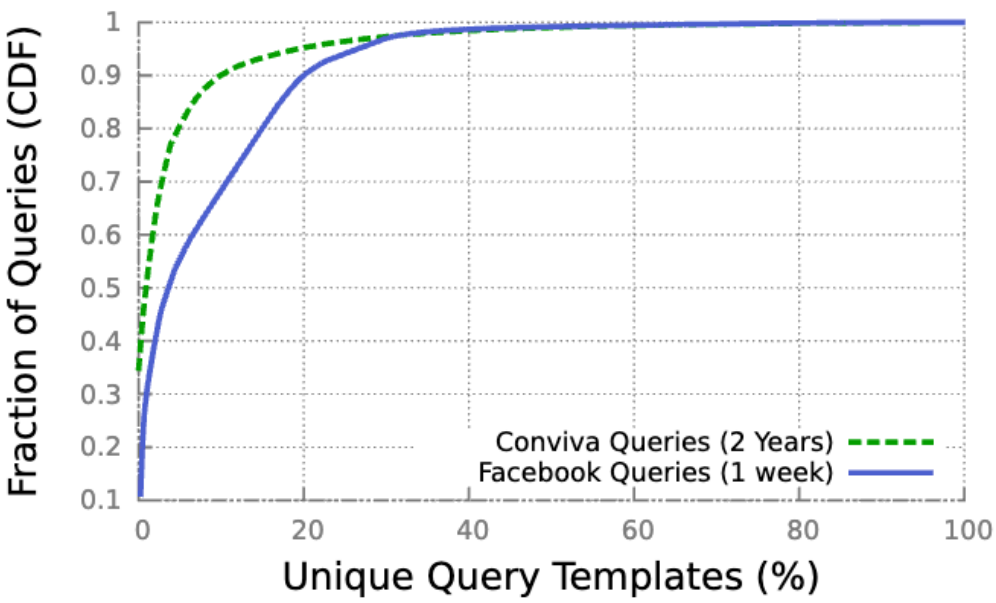


Figure 1. Taxonomy of workload models.

-
- Nice way to think about where their contribution lies, more a spectrum than a taxonomy
- Predictable Queries:

    - can use sketches, also histograms, wavelets
    - Detour for wavelets:
        - Basic idea: decompose a data distribution into frequency components
            - keep the top-k wavelet coefficients and can reconstruct approximate answers to range queries or histograms.
            - great for point and range aggregate queries but hard to update and don't generalize to arbitrary GROUP BY.
- Predictable Query Predicates:
    - if 5% of queries have `city = New York`, future queries will also include same predicate at same frequency
- Predictable QCS
    - If 5% of queries filtered on city, future queries will also group or filter on city at same frequency
    - AQUA from Bell Labs also uses a similar idea
        - but supports a single QCS
        - always reads entire QCS
- Unpredictable
    - Can't do much more than scanning in a random order and doing online aggregation
- Experimental Motivation:
    - 10% of QCSs cover 90% of workloads
    - 100s of QCSs cover 10s of 1000s of queries



Overview

- Offline Sampling Module
    - Per QCSs, do reservoir sampling
- Runtime Sample Selection
    - Pick the right QCS
    - Use a so-called Error Latency Profile
        - characterizing the rate of decrease of error as size of sample increase
- UX:

```
SELECT COUNT(*)
FROM Sessions
WHERE Genre = ‘western’
GROUP BY OS
ERROR WITHIN 10% AT CONFIDENCE 95%
```

or

```
SELECT COUNT(*)
FROM Sessions
WHERE Genre = ‘western’
GROUP BY OS
WITHIN 5 SECONDS
```

- No support for joins or nested SQL queries
  - "*we find that this is usually not a hindrance. is is because any query involving nested queries or joins can be flattened to run on the underlying data.*" Umm.. OK!

Sample Creation

- Let's start with some formalism
- Table T, Query Q, time and error bounds t, e
- n: number of rows that can be accessed in time t
  - Estimated later using ELP
- ϕ a QCS for Q; $x \in \phi$
- $D(\phi)$ set of unique x combinations for ϕ, i.e., the groups of ϕ
- T_x, S_x rows in T or sample S satisfying x
- $S(\phi, K)$, stratified sample along ϕ, each x capped by K
  - Pictorial illustration:

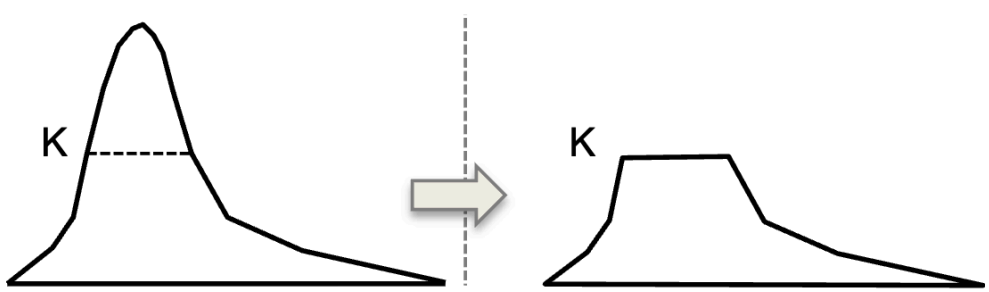
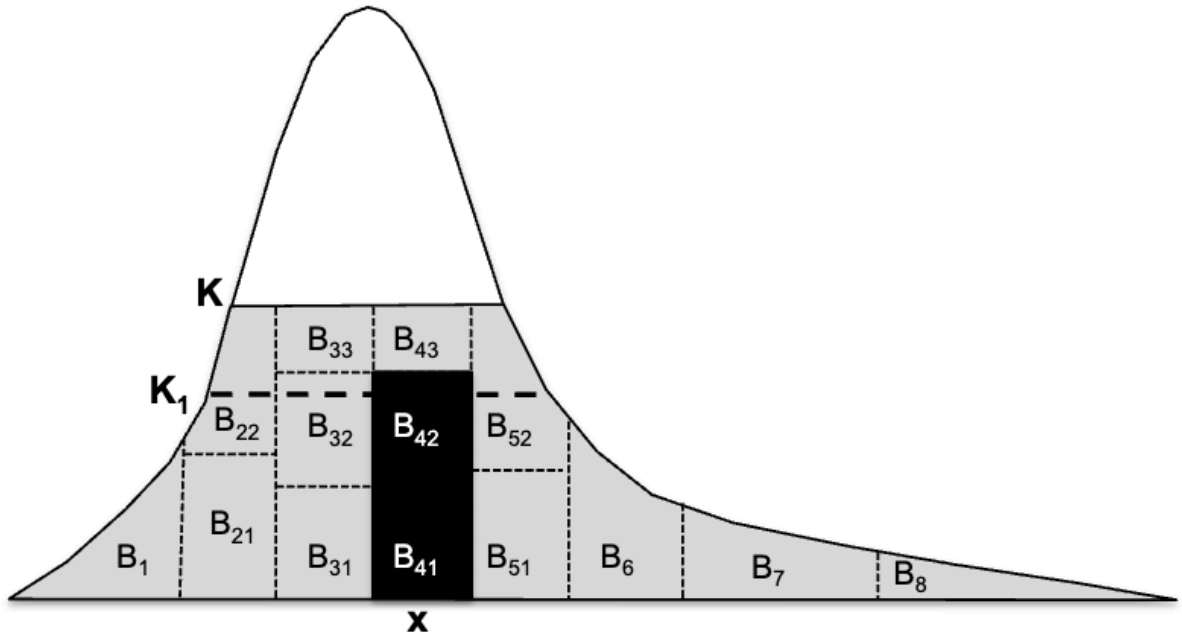


Figure 4. Example of a stratified sample associated with a set of columns, ϕ.

  - Basic idea: integrate area under the curve for various K, pick one for which area is equal to n
  - K influences standard error (width of CI) as $O(\sqrt{K})$



- n could also vary; some queries require fewer tuples, so we can break down samples per group into chunks, retrieve only those necessary for query, as in example above

- $\Delta(\phi, M)$: sparsity: number of groups x having size less than M
- Optimization problem:
  - A stratified sample on ϕ is useful if:
    - $\Delta(\phi, M)$ is small
      - How to set M? Unclear
    - it happens often in queries
  - It's a bit strange overall

$$G = \sum_j p_j \cdot y_j \cdot \Delta(q_j, M) \tag{1}$$

subject to

$$\sum_{i=1}^{m} |S(\phi_i, K)| \cdot z_i \leq \mathbb{C} \tag{2}$$

and

$$\forall\ j: \quad y_j \leq \max_{i:\phi_i \subseteq q_j \cup i:\phi_i \supset q_j} \left(z_i \min 1, \frac{|D(\phi_i)|}{|D(q_j)|}\right) \tag{3}$$

  - Two parts make sense (2) and (3)
    - (2) Storage is capped at C for those QCSs stored
    - (3) Each query is covered partially by QCSs that don't subsume it to account for, e.g., a uniform random sample partially helping a query with a specific WHERE clause
  - (1) does not make much sense to me; instead I would try to just do $\sum_j p_j y_j$.

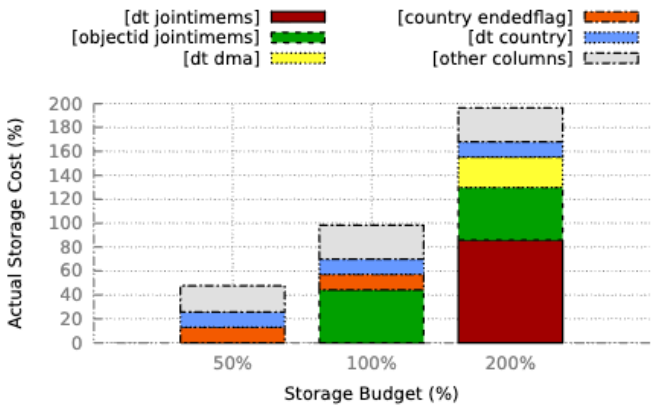
- Other issues
  - M = K = 100K. "Settings do well"
    - Not clear how to set this for new workloads
  - Why only give K for a QCS or none at all?
  - Can't partial covering cause issues? Bias?

Runtime

- **Stats Gathering** Executes query on small subsamples of one or or more samples to gather statistics "about the query's selectivity, complexity and the underlying distribution of its inputs.
  - For stats generation:
    - Pick smallest QCS that covers q
    - Otherwise, various use in-memory samples and estimate *selectivity* of QCSs
      - defined as fraction of applicable tuples compared to total tuples read
- **Picking Samples**
  - Error profile: plot error based on selectivity, number of records read, and statistical error formulae
    - Basically: query on a tiny *subsample* of each candidate QCS (e.g., 1% of the sample) to estimate selectivity.
    - It then uses closed-form statistical formulas (CLT-based) to project: "if I read N rows from this QCS, what error would I get?"
  - Latency profile: basically linearly based on number of records read
  - eg.

Evaluation

- Case studies on what is stored



(a) Biased Samples (Conviva)

- Latency for various data sizes
- Comparison of error for fixed latency for uniform, single, QCS
- etc.